Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

General

Enabled

Description

Log Generator details

Plain text Preview

- [ ] Discard old builds ?
- [ ] GitHub project
- [ ] This project is parameterized ?
- [ ] Throttle builds ?
- [ ] Execute concurrent builds if necessary ?

Advanced

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

- (•) None
- ( ) Git ?

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- [ ] Trigger builds remotely (e.g., from scripts) ?
- [ ] Build after other projects are built ?
- [ ] Build periodically ?
- [ ] GitHub hook trigger for GITScm polling ?
- [ ] Poll SCM ?

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

- [x] Delete workspace before build starts

  Advanced

- [ ] Use secret text(s) or file(s) ?
- [ ] Send files or execute commands over SSH before the build starts ?
- [ ] Send files or execute commands over SSH after the build runs ?
- [x] Add timestamps to the Console Output
- [ ] Inspect build log for published build scans
- [ ] Terminate a build if it's stuck
- [ ] With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

See the list of available environment variables

```bash
#!/bin/bash
set -e

APP_NAME="log-generator"
VERSION="1.0.0"

echo "🚀 Creating project structure..."

mkdir -p ${APP_NAME}/src/main/java/com/example
cd ${APP_NAME}

echo "📄 Creating Java log generator..."

cat << 'EOF' > src/main/java/com/example/LogGenerator.java
package com.example;

import java.io.FileWriter;
import java.io.PrintWriter;
import java.time.LocalDateTime;
import java.util.Random;
import java.util.UUID;

public class LogGenerator {

    private static final Random random = new Random();
    private static final String SERVICE_NAME = "log-generator-service";
    private static final String ENV =
            System.getenv().getOrDefault("APP_ENV", "dev");
    private static final String LOG_FILE =
            System.getenv().getOrDefault("LOG_FILE", "app.log");

    public static void main(String[] args) throws Exception {
        while (true) {
            String requestId = UUID.randomUUID().toString();
            long latency = random.nextInt(900) + 100;
            int logType = random.nextInt(10);

            if (logType <= 5) {
                writeLog("INFO","Request processed successfully",requestId,latency,null);
            } else if (logType <= 7) {
                writeLog("WARN","High response time detected",requestId,latency,"threshold=800ms");
            } else {
                writeLog("ERROR","Unhandled exception occurred",requestId,null,"Database timeout");
            }

            Thread.sleep(1000);
        }
    }

    private static void writeLog(String level, String message,
                                 String requestId, Long latency, String error)
            throws Exception {

        String log = String.format(
            "{ \"timestamp\": \"%s\", \"level\": \"%s\", \"service\": \"%s\", \"env\": \"%s\", " +
            "\"requestId\": \"%s\", \"message\": \"%s\", \"latency_ms\": %s, \"error\": %s }",
            LocalDateTime.now(), level, SERVICE_NAME, ENV,
            requestId, message,
            latency == null ? "null" : latency,
            error == null ? "null" : "\"" + error + "\""
        );

        try (PrintWriter out = new PrintWriter(new FileWriter(LOG_FILE, true))) {
            out.println(log);
        }

        System.out.println(log);
    }
}
EOF
```

```
echo "📄 Creating pom.xml..."

cat << EOF > pom.xml
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>${APP_NAME}</artifactId>
  <version>${VERSION}</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.3.0</version>
        <configuration>
          <archive>
            <manifest>
              <mainClass>com.example.LogGenerator</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
EOF

echo "🔨 Building JAR..."
mvn clean package

echo "✅ Build complete"
echo "📦 Artifact: target/${APP_NAME}-${VERSION}.jar"
```

Advanced

+ Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Send build artifacts over SSH ?

SSH Publishers

SSH Server

Name ?

worker

Advanced

Transfers

Transfer Set

Source files ?

log-generator/target/log-generator-1.0.0.jar

Remove prefix ?

log-generator/target

Remote directory ?

/

Exec command ?

```
cat << 'EOF' >/root/t1.sh
#!/bin/bash
pkill -f log-generator

APP_ENV=prod nohup /usr/bin/java -jar /root/log-generator-1.0.0.jar > /root/app.log 2>&1 &

sleep 2

EOF

chmod +x /root/t1.sh

sh /root/t1.sh
```

All of the transfer fields (except for Exec timeout) support substitution of Jenkins environment variables

Advanced

+ Add Transfer Set

+ Add Server

Advanced

+ Add post-build action

Save Apply